建立庫存監控模組程式文件

庫存監控模組-實作文件.md

我的選擇：FR-01 產品管理 + FR-02 庫存監控模組

選擇理由

從 8 大模組中選擇這兩個，原因：

考量	說明
資料就緒	直接沿用 `classicmodels.products`，110 筆真實資料，不需造假
最低相依	不需先建供應商、採購申請等表才能運行
業務核心	低庫存警示（S24_2000 僅剩 **15 件**）正是整個採購流程的觸發點
視覺豐富	Dashboard + 表格 + 篩選，能展示 Angular 完整互動能力

文件內容摘要

後端（Spring Boot 3.2 + Data JPA + MySQL）

- **`Product.java`** — 對應 `products` 表的 Entity
- **`StockThreshold.java`** — 新增的安全水位 Entity
- **`ProductRepository`** — 含低庫存 JPQL Query、Dashboard 統計聚合查詢
- **`ProductService`** — 產品 CRUD + 低庫存警示 + Dashboard 統計
- **`ProductController`** — 11 支 REST API（含篩選、低庫存、Dashboard）
- **`GlobalExceptionHandler`** — 統一 404 / 400 / Validation 錯誤格式

前端（Angular 17 Standalone）

Component	功能
`DashboardComponent`	4 格統計卡片 + 產品線庫存統計表 + 緊急補貨清單
`ProductListComponent`	關鍵字搜尋 + 產品線/供應商篩選 + 毛利率計算
`LowStockAlertComponent`	低庫存清單 + 安全水位 inline 編輯

新增 SQL

`stock_thresholds` 資料表 + 5 筆警示產品的預設水位設定

FR-02 庫存監控模組 — 完整實作文件

Product Inventory Monitoring Module — Full Implementation Guide

項目	說明
選用模組	FR-01 產品管理 + FR-02 庫存監控
選擇理由	使用現有 `classicmodels.products` 表即可運作；高優先等級需求；為其他採購模組的根基；低庫存警示具立即業務價值
後端技術	Spring Boot 3.2 + Spring Data JPA + MySQL
前端技術	Angular 17 (Standalone Components)
資料庫	MySQL — classicmodels（沿用） + 新增 `stock_thresholds`

選擇說明

從 8 大模組中，選擇 **FR-01 產品管理 + FR-02 庫存監控** 作為本期實作，理由：

1. **資料就緒**：直接使用 `classicmodels.products` 110 筆產品資料，無需額外資料準備
2. **高優先等級**：FR-01-01、FR-02-01、FR-02-02、FR-02-03 均為「高」優先
3. **獨立可運行**：不依賴採購申請、供應商等其他模組即可完整展示
4. **核心基礎**：庫存警示是觸發整個採購流程的第一步
5. **視覺豐富**：儀表板、低庫存警示表格、產品線統計圖具備良好 Demo 效果

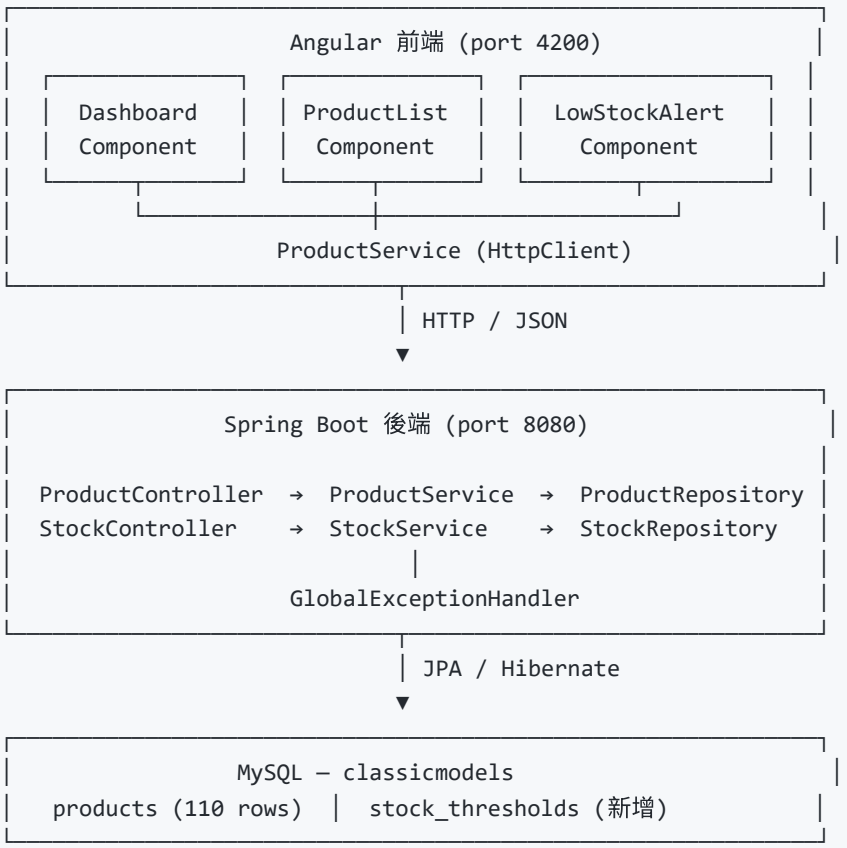
實作功能範圍

需求編號	功能	實作
FR-01-01	產品 CRUD	✅
FR-01-02	依產品線、供應商篩選	✅
FR-01-03	關鍵字搜尋	✅
FR-02-01	即時庫存顯示	✅
FR-02-02	設定安全水位	✅
FR-02-03	低庫存警示列表	✅
FR-02-05	儀表板（產品線庫存統計）	✅

目錄

---

1. 系統架構圖



---

2. 資料庫設計

2.1 沿用資料表 — `products`

```sql
-- classicmodels 原始資料表（不修改結構）
CREATE TABLE products (
    productCode       VARCHAR(15)  NOT NULL PRIMARY KEY,
    productName       VARCHAR(70)  NOT NULL,
    productLine       VARCHAR(50)  NOT NULL,
    productScale      VARCHAR(10)  NOT NULL,
    productVendor     VARCHAR(50)  NOT NULL,
    productDescription TEXT        NOT NULL,
    quantityInStock   SMALLINT     NOT NULL,
    buyPrice          DOUBLE       NOT NULL,
    MSRP              DOUBLE       NOT NULL
);
```

2.2 新增資料表 — `stock_thresholds`

```sql
CREATE TABLE stock_thresholds (
    id              INT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
```

```
    product_code     VARCHAR(15)   NOT NULL UNIQUE,
    min_quantity     INT           NOT NULL DEFAULT 200,
    reorder_quantity INT           NOT NULL DEFAULT 500,
    updated_at       DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
                                   ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_threshold_product
        FOREIGN KEY (product_code) REFERENCES products(productCode)
        ON DELETE CASCADE
);
```

2.3 ER 圖

```
products (PK: productCode)
    │
    │ 1:1（可無 threshold）
    ▼
stock_thresholds (FK: product_code → products.productCode)
```

---

3. 後端 Spring Boot 實作

3.1 pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
    </parent>

    <groupId>com.classicmodels</groupId>
    <artifactId>inventory-api</artifactId>
    <version>1.0.0</version>
    <name>inventory-api</name>
    <description>庫存監控模組 REST API</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Web / REST -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
```

```
        <!-- JPA + Hibernate -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- MySQL Driver -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Bean Validation -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- Test -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

3.2 application.properties

```
# src/main/resources/application.properties

# ── 伺服器 ──────────────────────────────────────
server.port=8080

# ── MySQL 資料來源 ──────────────────────────────
spring.datasource.url=jdbc:mysql://localhost:3306/classicmodels?
useSSL=false&serverTimezone=Asia/Taipei&characterEncoding=UTF-8
spring.datasource.username=root
spring.datasource.password=your_password_here
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# ── JPA / Hibernate ─────────────────────────────
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

```
# validate：驗證 Entity 對應；update：自動建立 stock_thresholds
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true

# ── 日誌 ──────────────────────────────────────
logging.level.com.classicmodels=INFO
```

3.3 Entity 實體類

Product.java

```java
// src/main/java/com/classicmodels/inventory/model/Product.java
package com.classicmodels.inventory.model;

import jakarta.persistence.*;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @Column(name = "productCode", length = 15)
    private String productCode;

    @Column(name = "productName", nullable = false, length = 70)
    private String productName;

    @Column(name = "productLine", nullable = false, length = 50)
    private String productLine;

    @Column(name = "productScale", nullable = false, length = 10)
    private String productScale;

    @Column(name = "productVendor", nullable = false, length = 50)
    private String productVendor;

    @Column(name = "productDescription", nullable = false, columnDefinition = "TEXT")
    private String productDescription;

    @Column(name = "quantityInStock", nullable = false)
    private int quantityInStock;

    @Column(name = "buyPrice", nullable = false)
    private double buyPrice;

    @Column(name = "MSRP", nullable = false)
    private double msrp;

    // ── Getters & Setters ──────────────────────
    public String getProductCode()                  { return productCode; }
    public void   setProductCode(String productCode)    { this.productCode = productCode; }

    public String getProductName()                  { return productName; }
```

```
    public void   setProductName(String productName)     { this.productName = productName; }

    public String getProductLine()                       { return productLine; }
    public void   setProductLine(String productLine)     { this.productLine = productLine; }

    public String getProductScale()                      { return productScale; }
    public void   setProductScale(String productScale)   { this.productScale = productScale; }

    public String getProductVendor()                     { return productVendor; }
    public void   setProductVendor(String productVendor) { this.productVendor = productVendor; }

    public String getProductDescription()                       { return productDescription; }
    public void   setProductDescription(String productDescription) { this.productDescription =
productDescription; }

    public int  getQuantityInStock()                 { return quantityInStock; }
    public void setQuantityInStock(int quantityInStock) { this.quantityInStock = quantityInStock; }

    public double getBuyPrice()              { return buyPrice; }
    public void   setBuyPrice(double buyPrice) { this.buyPrice = buyPrice; }

    public double getMsrp()            { return msrp; }
    public void   setMsrp(double msrp) { this.msrp = msrp; }
}
```

StockThreshold.java

```
// src/main/java/com/classicmodels/inventory/model/StockThreshold.java
package com.classicmodels.inventory.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "stock_thresholds")
public class StockThreshold {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "product_code", nullable = false, unique = true, length = 15)
    private String productCode;

    @Column(name = "min_quantity", nullable = false)
    private int minQuantity = 200;

    @Column(name = "reorder_quantity", nullable = false)
    private int reorderQuantity = 500;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    @PreUpdate
    public void onUpdate() {
```

```
        this.updatedAt = LocalDateTime.now();
    }

    // ── Getters & Setters ──────────────────────────
    public Integer getId()           { return id; }
    public void    setId(Integer id) { this.id = id; }

    public String getProductCode()                   { return productCode; }
    public void   setProductCode(String productCode) { this.productCode = productCode; }

    public int  getMinQuantity()                 { return minQuantity; }
    public void setMinQuantity(int minQuantity)  { this.minQuantity = minQuantity; }

    public int  getReorderQuantity()                     { return reorderQuantity; }
    public void setReorderQuantity(int reorderQuantity)  { this.reorderQuantity = reorderQuantity;
}

    public LocalDateTime getUpdatedAt()                   { return updatedAt; }
    public void          setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }
}
```

3.4 Repository 介面

ProductRepository.java

```
// src/main/java/com/classicmodels/inventory/repository/ProductRepository.java
package com.classicmodels.inventory.repository;

import com.classicmodels.inventory.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;

public interface ProductRepository extends JpaRepository<Product, String> {

    // FR-01-02：依產品線篩選
    List<Product> findByProductLine(String productLine);

    // FR-01-02：依供應商篩選
    List<Product> findByProductVendor(String productVendor);

    // FR-01-03：關鍵字搜尋（productName 或 productDescription）
    @Query("SELECT p FROM Product p WHERE " +
           "LOWER(p.productName) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
           "LOWER(p.productDescription) LIKE LOWER(CONCAT('%', :keyword, '%'))")
    List<Product> searchByKeyword(@Param("keyword") String keyword);

    // FR-02-03：低庫存警示 — 取得低於安全水位的產品
    // 使用 JOIN stock_thresholds，若無 threshold 則以預設值 200 比較
    @Query("SELECT p FROM Product p WHERE p.quantityInStock < " +
           "(SELECT COALESCE(st.minQuantity, 200) FROM StockThreshold st " +
           " WHERE st.productCode = p.productCode)")
```

```
    List<Product> findLowStockProducts();

    // FR-02-05：各產品線庫存統計（Dashboard 用）
    @Query("SELECT p.productLine, SUM(p.quantityInStock), COUNT(p), MIN(p.quantityInStock) " +
           "FROM Product p GROUP BY p.productLine ORDER BY SUM(p.quantityInStock) DESC")
    List<Object[]> getStockSummaryByProductLine();

    // 取得所有不重複產品線
    @Query("SELECT DISTINCT p.productLine FROM Product p ORDER BY p.productLine")
    List<String> findDistinctProductLines();

    // 取得所有不重複供應商
    @Query("SELECT DISTINCT p.productVendor FROM Product p ORDER BY p.productVendor")
    List<String> findDistinctVendors();
}
```

StockThresholdRepository.java

```
// src/main/java/com/classicmodels/inventory/repository/StockThresholdRepository.java
package com.classicmodels.inventory.repository;

import com.classicmodels.inventory.model.StockThreshold;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface StockThresholdRepository extends JpaRepository<StockThreshold, Integer> {

    Optional<StockThreshold> findByProductCode(String productCode);
}
```

3.5 DTO

ProductDTO.java

```
// src/main/java/com/classicmodels/inventory/dto/ProductDTO.java
package com.classicmodels.inventory.dto;

import jakarta.validation.constraints.*;

public class ProductDTO {

    @NotBlank
    @Size(max = 15)
    private String productCode;

    @NotBlank
    @Size(max = 70)
    private String productName;

    @NotBlank
    private String productLine;
```

```
    @NotBlank
    private String productScale;

    @NotBlank
    private String productVendor;

    @NotBlank
    private String productDescription;

    @Min(0)
    private int quantityInStock;

    @DecimalMin("0.0")
    private double buyPrice;

    @DecimalMin("0.0")
    private double msrp;

    // ── Getters & Setters ──────────────────────────
    public String getProductCode()                        { return productCode; }
    public void   setProductCode(String productCode)      { this.productCode = productCode; }
    public String getProductName()                        { return productName; }
    public void   setProductName(String productName)      { this.productName = productName; }
    public String getProductLine()                        { return productLine; }
    public void   setProductLine(String productLine)      { this.productLine = productLine; }
    public String getProductScale()                       { return productScale; }
    public void   setProductScale(String productScale)    { this.productScale = productScale; }
    public String getProductVendor()                      { return productVendor; }
    public void   setProductVendor(String productVendor)  { this.productVendor = productVendor; }
    public String getProductDescription()                 { return productDescription; }
    public void   setProductDescription(String d)         { this.productDescription = d; }
    public int    getQuantityInStock()                    { return quantityInStock; }
    public void   setQuantityInStock(int quantityInStock) { this.quantityInStock =
quantityInStock; }
    public double getBuyPrice()                           { return buyPrice; }
    public void   setBuyPrice(double buyPrice)            { this.buyPrice = buyPrice; }
    public double getMsrp()                               { return msrp; }
    public void   setMsrp(double msrp)                   { this.msrp = msrp; }
}
```

`StockSummaryDTO.java`（Dashboard 回傳用）

```
// src/main/java/com/classicmodels/inventory/dto/StockSummaryDTO.java
package com.classicmodels.inventory.dto;

public class StockSummaryDTO {

    private String productLine;
    private long   totalStock;
    private long   productCount;
    private int    minStock;

    public StockSummaryDTO(String productLine, long totalStock, long productCount, int minStock) {
        this.productLine  = productLine;
        this.totalStock   = totalStock;
```

```
        this.productCount = productCount;
        this.minStock     = minStock;
    }

    public String getProductLine()  { return productLine; }
    public long   getTotalStock()   { return totalStock; }
    public long   getProductCount() { return productCount; }
    public int    getMinStock()     { return minStock; }
}
```

`ApiResponse.java`（統一回傳格式）

```
// src/main/java/com/classicmodels/inventory/dto/ApiResponse.java
package com.classicmodels.inventory.dto;

public class ApiResponse<T> {

    private boolean success;
    private String  message;
    private T       data;

    public static <T> ApiResponse<T> ok(T data) {
        ApiResponse<T> res = new ApiResponse<>();
        res.success = true;
        res.message = "success";
        res.data    = data;
        return res;
    }

    public static <T> ApiResponse<T> ok(String message, T data) {
        ApiResponse<T> res = new ApiResponse<>();
        res.success = true;
        res.message = message;
        res.data    = data;
        return res;
    }

    public boolean isSuccess() { return success; }
    public String  getMessage() { return message; }
    public T       getData()    { return data; }
}
```

3.6 Service 商業邏輯

`ProductService.java`

```
// src/main/java/com/classicmodels/inventory/service/ProductService.java
package com.classicmodels.inventory.service;

import com.classicmodels.inventory.dto.ProductDTO;
import com.classicmodels.inventory.dto.StockSummaryDTO;
import com.classicmodels.inventory.exception.ResourceNotFoundException;
```

```java
import com.classicmodels.inventory.model.Product;
import com.classicmodels.inventory.repository.ProductRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
@Transactional(readOnly = true)
public class ProductService {

    private final ProductRepository productRepo;

    public ProductService(ProductRepository productRepo) {
        this.productRepo = productRepo;
    }

    // ── 查詢全部 ──────────────────────────────────
    public List<Product> findAll() {
        return productRepo.findAll();
    }

    // ── 依 productCode 查詢 ──────────────────────
    public Product findById(String productCode) {
        return productRepo.findById(productCode)
                .orElseThrow(() -> new ResourceNotFoundException(
                        "Product not found: " + productCode));
    }

    // ── 依產品線篩選 ──────────────────────────────
    public List<Product> findByProductLine(String productLine) {
        return productRepo.findByProductLine(productLine);
    }

    // ── 依供應商篩選 ──────────────────────────────
    public List<Product> findByVendor(String vendor) {
        return productRepo.findByProductVendor(vendor);
    }

    // ── 關鍵字搜尋 ────────────────────────────────
    public List<Product> search(String keyword) {
        return productRepo.searchByKeyword(keyword);
    }

    // ── 低庫存警示清單 ────────────────────────────
    public List<Product> findLowStockProducts() {
        return productRepo.findLowStockProducts();
    }

    // ── Dashboard：各產品線庫存統計 ──────────────
    public List<StockSummaryDTO> getStockSummary() {
        return productRepo.getStockSummaryByProductLine()
                .stream()
                .map(row -> new StockSummaryDTO(
                        (String) row[0],
                        ((Number) row[1]).longValue(),
                        ((Number) row[2]).longValue(),
```

```
                    ((Number) row[3]).intValue()
            ))
            .collect(Collectors.toList());
    }

    // ── 取得所有產品線 ──────────────────────────
    public List<String> findAllProductLines() {
        return productRepo.findDistinctProductLines();
    }

    // ── 取得所有供應商 ──────────────────────────
    public List<String> findAllVendors() {
        return productRepo.findDistinctVendors();
    }

    // ── 新增產品 ────────────────────────────────
    @Transactional
    public Product create(ProductDTO dto) {
        if (productRepo.existsById(dto.getProductCode())) {
            throw new IllegalArgumentException(
                    "Product code already exists: " + dto.getProductCode());
        }
        return productRepo.save(mapToEntity(dto));
    }

    // ── 更新產品 ────────────────────────────────
    @Transactional
    public Product update(String productCode, ProductDTO dto) {
        Product existing = findById(productCode);
        existing.setProductName(dto.getProductName());
        existing.setProductLine(dto.getProductLine());
        existing.setProductScale(dto.getProductScale());
        existing.setProductVendor(dto.getProductVendor());
        existing.setProductDescription(dto.getProductDescription());
        existing.setQuantityInStock(dto.getQuantityInStock());
        existing.setBuyPrice(dto.getBuyPrice());
        existing.setMsrp(dto.getMsrp());
        return productRepo.save(existing);
    }

    // ── 刪除產品 ────────────────────────────────
    @Transactional
    public void delete(String productCode) {
        Product existing = findById(productCode);
        productRepo.delete(existing);
    }

    // ── 私有：DTO → Entity ──────────────────────
    private Product mapToEntity(ProductDTO dto) {
        Product p = new Product();
        p.setProductCode(dto.getProductCode());
        p.setProductName(dto.getProductName());
        p.setProductLine(dto.getProductLine());
        p.setProductScale(dto.getProductScale());
        p.setProductVendor(dto.getProductVendor());
        p.setProductDescription(dto.getProductDescription());
        p.setQuantityInStock(dto.getQuantityInStock());
        p.setBuyPrice(dto.getBuyPrice());
```

```
        p.setMsrp(dto.getMsrp());
        return p;
    }
}
```

StockThresholdService.java

```
// src/main/java/com/classicmodels/inventory/service/StockThresholdService.java
package com.classicmodels.inventory.service;

import com.classicmodels.inventory.exception.ResourceNotFoundException;
import com.classicmodels.inventory.model.StockThreshold;
import com.classicmodels.inventory.repository.ProductRepository;
import com.classicmodels.inventory.repository.StockThresholdRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
public class StockThresholdService {

    private final StockThresholdRepository thresholdRepo;
    private final ProductRepository        productRepo;

    public StockThresholdService(StockThresholdRepository thresholdRepo,
                                 ProductRepository productRepo) {
        this.thresholdRepo = thresholdRepo;
        this.productRepo   = productRepo;
    }

    // ── 查詢特定產品安全水位 ──────────────────────
    public StockThreshold findByProductCode(String productCode) {
        return thresholdRepo.findByProductCode(productCode)
                .orElseThrow(() -> new ResourceNotFoundException(
                        "Threshold not found for product: " + productCode));
    }

    // ── 新增或更新安全水位 ────────────────────────
    @Transactional
    public StockThreshold upsert(String productCode, int minQty, int reorderQty) {
        if (minQty <= 0 || reorderQty <= 0) {
            throw new IllegalArgumentException("Quantity values must be greater than 0");
        }
        if (!productRepo.existsById(productCode)) {
            throw new ResourceNotFoundException("Product not found: " + productCode);
        }

        StockThreshold threshold = thresholdRepo.findByProductCode(productCode)
                .orElse(new StockThreshold());

        threshold.setProductCode(productCode);
        threshold.setMinQuantity(minQty);
        threshold.setReorderQuantity(reorderQty);
        return thresholdRepo.save(threshold);
```

```
    }
}
```

3.7 Controller REST API

ProductController.java

```
// src/main/java/com/classicmodels/inventory/controller/ProductController.java
package com.classicmodels.inventory.controller;

import com.classicmodels.inventory.dto.ApiResponse;
import com.classicmodels.inventory.dto.ProductDTO;
import com.classicmodels.inventory.service.ProductService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    // GET /api/products — 查詢全部（可加 ?productLine=&vendor=&keyword= 篩選）
    @GetMapping
    public ResponseEntity<ApiResponse<Object>> getAll(
            @RequestParam(required = false) String productLine,
            @RequestParam(required = false) String vendor,
            @RequestParam(required = false) String keyword) {

        Object result;
        if (keyword != null && !keyword.isBlank()) {
            result = productService.search(keyword);
        } else if (productLine != null && !productLine.isBlank()) {
            result = productService.findByProductLine(productLine);
        } else if (vendor != null && !vendor.isBlank()) {
            result = productService.findByVendor(vendor);
        } else {
            result = productService.findAll();
        }
        return ResponseEntity.ok(ApiResponse.ok(result));
    }

    // GET /api/products/{productCode} — 查詢單一產品
    @GetMapping("/{productCode}")
    public ResponseEntity<ApiResponse<Object>> getById(@PathVariable String productCode) {
        return ResponseEntity.ok(ApiResponse.ok(productService.findById(productCode)));
```

```
    }

    // GET /api/products/low-stock — 低庫存警示清單
    @GetMapping("/low-stock")
    public ResponseEntity<ApiResponse<Object>> getLowStock() {
        return ResponseEntity.ok(ApiResponse.ok(productService.findLowStockProducts()));
    }

    // GET /api/products/dashboard/summary — 儀表板統計
    @GetMapping("/dashboard/summary")
    public ResponseEntity<ApiResponse<Object>> getDashboardSummary() {
        return ResponseEntity.ok(ApiResponse.ok(productService.getStockSummary()));
    }

    // GET /api/products/filter/product-lines — 所有產品線
    @GetMapping("/filter/product-lines")
    public ResponseEntity<ApiResponse<List<String>>> getProductLines() {
        return ResponseEntity.ok(ApiResponse.ok(productService.findAllProductLines()));
    }

    // GET /api/products/filter/vendors — 所有供應商
    @GetMapping("/filter/vendors")
    public ResponseEntity<ApiResponse<List<String>>> getVendors() {
        return ResponseEntity.ok(ApiResponse.ok(productService.findAllVendors()));
    }

    // POST /api/products — 新增產品
    @PostMapping
    public ResponseEntity<ApiResponse<Object>> create(@Valid @RequestBody ProductDTO dto) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(ApiResponse.ok("Product created", productService.create(dto)));
    }

    // PUT /api/products/{productCode} — 更新產品
    @PutMapping("/{productCode}")
    public ResponseEntity<ApiResponse<Object>> update(
            @PathVariable String productCode,
            @Valid @RequestBody ProductDTO dto) {
        return ResponseEntity.ok(ApiResponse.ok("Product updated",
                productService.update(productCode, dto)));
    }

    // DELETE /api/products/{productCode} — 刪除產品
    @DeleteMapping("/{productCode}")
    public ResponseEntity<ApiResponse<Object>> delete(@PathVariable String productCode) {
        productService.delete(productCode);
        return ResponseEntity.ok(ApiResponse.ok("Product deleted", null));
    }
}
```

StockThresholdController.java

```
// src/main/java/com/classicmodels/inventory/controller/StockThresholdController.java
package com.classicmodels.inventory.controller;

import com.classicmodels.inventory.dto.ApiResponse;
```

```java
import com.classicmodels.inventory.service.StockThresholdService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/thresholds")
public class StockThresholdController {

    private final StockThresholdService thresholdService;

    public StockThresholdController(StockThresholdService thresholdService) {
        this.thresholdService = thresholdService;
    }

    // GET /api/thresholds/{productCode}
    @GetMapping("/{productCode}")
    public ResponseEntity<ApiResponse<Object>> getThreshold(@PathVariable String productCode) {
        return ResponseEntity.ok(ApiResponse.ok(thresholdService.findByProductCode(productCode)));
    }

    // PUT /api/thresholds/{productCode} — 新增或更新安全水位
    // Body: { "minQuantity": 200, "reorderQuantity": 500 }
    @PutMapping("/{productCode}")
    public ResponseEntity<ApiResponse<Object>> upsertThreshold(
            @PathVariable String productCode,
            @RequestBody Map<String, Integer> body) {

        int minQty     = body.getOrDefault("minQuantity", 200);
        int reorderQty = body.getOrDefault("reorderQuantity", 500);
        return ResponseEntity.ok(ApiResponse.ok("Threshold saved",
                thresholdService.upsert(productCode, minQty, reorderQty)));
    }
}
```

3.8 全域例外處理

```java
// src/main/java/com/classicmodels/inventory/exception/ResourceNotFoundException.java
package com.classicmodels.inventory.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

```java
// src/main/java/com/classicmodels/inventory/exception/GlobalExceptionHandler.java
package com.classicmodels.inventory.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
```

```java
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // 資源不存在 → 404
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex) {
        return buildError(HttpStatus.NOT_FOUND, "NOT_FOUND", ex.getMessage());
    }

    // 業務規則違反 → 400
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, Object>> handleBadRequest(IllegalArgumentException ex) {
        return buildError(HttpStatus.BAD_REQUEST, "BAD_REQUEST", ex.getMessage());
    }

    // Bean Validation 錯誤 → 400
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)
{
        Map<String, String> fieldErrors = new HashMap<>();
        for (FieldError fe : ex.getBindingResult().getFieldErrors()) {
            fieldErrors.put(fe.getField(), fe.getDefaultMessage());
        }
        Map<String, Object> body = new HashMap<>();
        body.put("success", false);
        body.put("errorCode", "VALIDATION_ERROR");
        body.put("errors", fieldErrors);
        return ResponseEntity.badRequest().body(body);
    }

    private ResponseEntity<Map<String, Object>> buildError(
            HttpStatus status, String errorCode, String message) {
        Map<String, Object> body = new HashMap<>();
        body.put("success", false);
        body.put("errorCode", errorCode);
        body.put("message", message);
        return ResponseEntity.status(status).body(body);
    }
}
```

3.9 CORS 設定 + 應用程式入口

```java
// src/main/java/com/classicmodels/inventory/config/CorsConfig.java
package com.classicmodels.inventory.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```java
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;

import java.util.List;

@Configuration
public class CorsConfig {

    @Bean
    public CorsFilter corsFilter() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("http://localhost:4200"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("*"));

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config);
        return new CorsFilter(source);
    }
}
```

```java
// src/main/java/com/classicmodels/inventory/InventoryApplication.java
package com.classicmodels.inventory;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class InventoryApplication {
    public static void main(String[] args) {
        SpringApplication.run(InventoryApplication.class, args);
    }
}
```

後端專案目錄結構

```
src/main/java/com/classicmodels/inventory/
├── InventoryApplication.java
├── config/
│   └── CorsConfig.java
├── controller/
│   ├── ProductController.java
│   └── StockThresholdController.java
├── dto/
│   ├── ApiResponse.java
│   ├── ProductDTO.java
│   └── StockSummaryDTO.java
├── exception/
│   ├── GlobalExceptionHandler.java
│   └── ResourceNotFoundException.java
├── model/
│   ├── Product.java
│   └── StockThreshold.java
├── repository/
```

```
│   ├── ProductRepository.java
│   └── StockThresholdRepository.java
└── service/
    ├── ProductService.java
    └── StockThresholdService.java
```

---

4. 前端 Angular 實作

4.1 專案結構

```
src/app/
├── app.component.ts              ← 根元件（含導覽列）
├── app.routes.ts                 ← 路由設定
├── models/
│   ├── product.model.ts
│   └── stock-summary.model.ts
├── services/
│   └── product.service.ts
└── components/
    ├── dashboard/
    │   └── dashboard.component.ts
    ├── product-list/
    │   ├── product-list.component.ts
    │   └── product-list.component.html
    └── low-stock-alert/
        ├── low-stock-alert.component.ts
        └── low-stock-alert.component.html
```

4.2 Model 介面

product.model.ts

```typescript
// src/app/models/product.model.ts
export interface Product {
  productCode: string;
  productName: string;
  productLine: string;
  productScale: string;
  productVendor: string;
  productDescription: string;
  quantityInStock: number;
  buyPrice: number;
  msrp: number;
}

export interface ApiResponse<T> {
  success: boolean;
  message: string;
```

```
  data: T;
}
```

stock-summary.model.ts

```
// src/app/models/stock-summary.model.ts
export interface StockSummary {
  productLine: string;
  totalStock: number;
  productCount: number;
  minStock: number;
}

export interface StockThreshold {
  id: number;
  productCode: string;
  minQuantity: number;
  reorderQuantity: number;
  updatedAt: string;
}
```

4.3 ProductService

```
// src/app/services/product.service.ts
import { Injectable } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable, map } from 'rxjs';
import { Product, ApiResponse } from '../models/product.model';
import { StockSummary, StockThreshold } from '../models/stock-summary.model';

@Injectable({ providedIn: 'root' })
export class ProductService {

  private readonly apiUrl = 'http://localhost:8080/api';

  constructor(private http: HttpClient) {}

  // ── 產品 CRUD ──────────────────────────────────────

  getProducts(filters?: { productLine?: string; vendor?: string; keyword?: string })
      : Observable<Product[]> {
    let params = new HttpParams();
    if (filters?.productLine) params = params.set('productLine', filters.productLine);
    if (filters?.vendor)      params = params.set('vendor', filters.vendor);
    if (filters?.keyword)     params = params.set('keyword', filters.keyword);
    return this.http.get<ApiResponse<Product[]>>(`${this.apiUrl}/products`, { params })
               .pipe(map(res => res.data));
  }

  getProductById(productCode: string): Observable<Product> {
    return this.http.get<ApiResponse<Product>>(`${this.apiUrl}/products/${productCode}`)
               .pipe(map(res => res.data));
```

```
  }

  createProduct(product: Product): Observable<Product> {
    return this.http.post<ApiResponse<Product>>(`${this.apiUrl}/products`, product)
               .pipe(map(res => res.data));
  }

  updateProduct(productCode: string, product: Product): Observable<Product> {
    return this.http.put<ApiResponse<Product>>(`${this.apiUrl}/products/${productCode}`, product)
               .pipe(map(res => res.data));
  }

  deleteProduct(productCode: string): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/products/${productCode}`);
  }

  // ── 庫存監控 ──────────────────────────────────────

  getLowStockProducts(): Observable<Product[]> {
    return this.http.get<ApiResponse<Product[]>>(`${this.apiUrl}/products/low-stock`)
               .pipe(map(res => res.data));
  }

  getDashboardSummary(): Observable<StockSummary[]> {
    return this.http.get<ApiResponse<StockSummary[]>>(`${this.apiUrl}/products/dashboard/summary`)
               .pipe(map(res => res.data));
  }

  // ── 篩選選項 ──────────────────────────────────────

  getProductLines(): Observable<string[]> {
    return this.http.get<ApiResponse<string[]>>(`${this.apiUrl}/products/filter/product-lines`)
               .pipe(map(res => res.data));
  }

  getVendors(): Observable<string[]> {
    return this.http.get<ApiResponse<string[]>>(`${this.apiUrl}/products/filter/vendors`)
               .pipe(map(res => res.data));
  }

  // ── 安全水位 ──────────────────────────────────────

  getThreshold(productCode: string): Observable<StockThreshold> {
    return this.http.get<ApiResponse<StockThreshold>>(`${this.apiUrl}/thresholds/${productCode}`)
               .pipe(map(res => res.data));
  }

  saveThreshold(productCode: string, minQuantity: number, reorderQuantity: number)
      : Observable<StockThreshold> {
    return this.http.put<ApiResponse<StockThreshold>>(
      `${this.apiUrl}/thresholds/${productCode}`,
      { minQuantity, reorderQuantity }
    ).pipe(map(res => res.data));
  }
}
```

4.4 Dashboard Component

```
// src/app/components/dashboard/dashboard.component.ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router';
import { ProductService } from '../../services/product.service';
import { StockSummary } from '../../models/stock-summary.model';
import { Product } from '../../models/product.model';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule, RouterModule],
  template: `
    <div class="dashboard-container">
      <h2>庫存監控儀表板</h2>

      <!-- 統計摘要卡片 -->
      <div class="summary-cards">
        <div class="card">
          <div class="card-value">{{ totalProducts }}</div>
          <div class="card-label">總產品數</div>
        </div>
        <div class="card">
          <div class="card-value">{{ totalStock | number }}</div>
          <div class="card-label">總庫存量</div>
        </div>
        <div class="card alert-card">
          <div class="card-value text-danger">{{ lowStockCount }}</div>
          <div class="card-label">低庫存警示</div>
        </div>
        <div class="card">
          <div class="card-value">{{ productLineCount }}</div>
          <div class="card-label">產品線數量</div>
        </div>
      </div>

      <!-- 各產品線庫存統計表 -->
      <h3>各產品線庫存統計</h3>
      <table class="data-table" *ngIf="summaries.length > 0; else loadingTpl">
        <thead>
          <tr>
            <th>產品線</th>
            <th>產品數</th>
            <th>總庫存</th>
            <th>最低庫存</th>
            <th>庫存狀態</th>
          </tr>
        </thead>
        <tbody>
          <tr *ngFor="let s of summaries">
            <td>{{ s.productLine }}</td>
            <td class="text-center">{{ s.productCount }}</td>
            <td class="text-right">{{ s.totalStock | number }}</td>
            <td class="text-right" [class.text-danger]="s.minStock < 200">
              {{ s.minStock | number }}
```

```
          </td>
          <td>
            <span class="badge" [class.badge-danger]="s.minStock < 100"
                                [class.badge-warning]="s.minStock >= 100 && s.minStock < 500"
                                [class.badge-success]="s.minStock >= 500">
              {{ getStockStatusLabel(s.minStock) }}
            </span>
          </td>
        </tr>
      </tbody>
    </table>

    <!-- 低庫存警示 -->
    <h3>緊急補貨清單 <span class="badge badge-danger">{{ lowStockProducts.length }}</span></h3>
    <table class="data-table" *ngIf="lowStockProducts.length > 0">
      <thead>
        <tr>
          <th>產品代碼</th>
          <th>產品名稱</th>
          <th>產品線</th>
          <th>供應商</th>
          <th>庫存數量</th>
          <th>操作</th>
        </tr>
      </thead>
      <tbody>
        <tr *ngFor="let p of lowStockProducts">
          <td>{{ p.productCode }}</td>
          <td>{{ p.productName }}</td>
          <td>{{ p.productLine }}</td>
          <td>{{ p.productVendor }}</td>
          <td class="text-right">
            <span class="text-danger fw-bold">{{ p.quantityInStock | number }}</span>
          </td>
          <td>
            <a [routerLink]="['/products', p.productCode]" class="btn btn-sm">
              查看
            </a>
          </td>
        </tr>
      </tbody>
    </table>
    <p *ngIf="lowStockProducts.length === 0" class="text-success">
      目前無低庫存警示產品
    </p>
  </div>

  <ng-template #loadingTpl>
    <p>載入中...</p>
  </ng-template>
`,
styles: [`
  .dashboard-container { padding: 1.5rem; }
  .summary-cards { display: flex; gap: 1rem; margin-bottom: 2rem; }
  .card {
    flex: 1; padding: 1.5rem; border-radius: 8px;
    background: #fff; box-shadow: 0 2px 8px rgba(0,0,0,0.1); text-align: center;
  }
```

```
    .alert-card { border-left: 4px solid #dc3545; }
    .card-value { font-size: 2rem; font-weight: bold; color: #343a40; }
    .card-label { color: #6c757d; margin-top: 0.5rem; }
    .data-table { width: 100%; border-collapse: collapse; margin-bottom: 2rem; }
    .data-table th, .data-table td { padding: 0.75rem; border-bottom: 1px solid #dee2e6; }
    .data-table th { background: #f8f9fa; font-weight: 600; }
    .text-center { text-align: center; }
    .text-right  { text-align: right; }
    .text-danger { color: #dc3545; }
    .text-success { color: #28a745; }
    .fw-bold { font-weight: bold; }
    .badge { padding: 0.3rem 0.6rem; border-radius: 4px; font-size: 0.8rem; color: #fff; }
    .badge-danger  { background: #dc3545; }
    .badge-warning { background: #ffc107; color: #000; }
    .badge-success { background: #28a745; }
    .btn { padding: 0.3rem 0.8rem; border: 1px solid #007bff; color: #007bff;
           border-radius: 4px; text-decoration: none; cursor: pointer; }
    .btn:hover { background: #007bff; color: #fff; }
    .btn-sm { font-size: 0.85rem; }
    h2 { color: #343a40; margin-bottom: 1.5rem; }
    h3 { color: #495057; margin: 1.5rem 0 0.75rem; }
  `]
})
export class DashboardComponent implements OnInit {

  summaries: StockSummary[]  = [];
  lowStockProducts: Product[] = [];

  get totalProducts()    { return this.summaries.reduce((s, r) => s + r.productCount, 0); }
  get totalStock()       { return this.summaries.reduce((s, r) => s + r.totalStock, 0); }
  get lowStockCount()    { return this.lowStockProducts.length; }
  get productLineCount() { return this.summaries.length; }

  constructor(private productService: ProductService) {}

  ngOnInit(): void {
    this.productService.getDashboardSummary().subscribe(data => this.summaries = data);
    this.productService.getLowStockProducts().subscribe(data => this.lowStockProducts = data);
  }

  getStockStatusLabel(minStock: number): string {
    if (minStock < 100)  return '緊急補貨';
    if (minStock < 500)  return '低庫存';
    return '正常';
  }
}
```

4.5 ProductList Component

```
// src/app/components/product-list/product-list.component.ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { RouterModule } from '@angular/router';
```

```
import { ProductService } from '../../services/product.service';
import { Product } from '../../models/product.model';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [CommonModule, FormsModule, RouterModule],
  templateUrl: './product-list.component.html',
  styles: [`
    .filter-bar { display: flex; gap: 0.75rem; margin-bottom: 1.5rem; flex-wrap: wrap; }
    .filter-bar input, .filter-bar select {
      padding: 0.5rem; border: 1px solid #ced4da; border-radius: 4px; min-width: 180px;
    }
    .filter-bar button {
      padding: 0.5rem 1rem; background: #007bff; color: #fff;
      border: none; border-radius: 4px; cursor: pointer;
    }
    .filter-bar .btn-reset {
      background: #6c757d;
    }
    .data-table { width: 100%; border-collapse: collapse; }
    .data-table th, .data-table td { padding: 0.75rem; border-bottom: 1px solid #dee2e6; }
    .data-table th { background: #f8f9fa; }
    .data-table tbody tr:hover { background: #f1f3f5; cursor: pointer; }
    .text-danger { color: #dc3545; font-weight: bold; }
    .page-title { margin-bottom: 1rem; color: #343a40; }
    .result-count { color: #6c757d; margin-bottom: 1rem; }
    .stock-cell { text-align: right; }
    .price-cell { text-align: right; }
  `]
})
export class ProductListComponent implements OnInit {

  products: Product[] = [];
  productLines: string[] = [];
  vendors: string[] = [];

  selectedLine  = '';
  selectedVendor = '';
  keyword       = '';

  constructor(private productService: ProductService) {}

  ngOnInit(): void {
    this.loadProducts();
    this.productService.getProductLines().subscribe(d => this.productLines = d);
    this.productService.getVendors().subscribe(d => this.vendors = d);
  }

  loadProducts(): void {
    const filters = {
      productLine: this.selectedLine   || undefined,
      vendor:      this.selectedVendor || undefined,
      keyword:     this.keyword        || undefined
    };
    this.productService.getProducts(filters).subscribe(data => this.products = data);
  }
```

```
  onSearch(): void  { this.loadProducts(); }

  onReset(): void {
    this.selectedLine   = '';
    this.selectedVendor = '';
    this.keyword        = '';
    this.loadProducts();
  }

  isLowStock(quantity: number): boolean { return quantity < 500; }

  calcMargin(buyPrice: number, msrp: number): string {
    if (msrp === 0) return '0%';
    return (((msrp - buyPrice) / msrp) * 100).toFixed(1) + '%';
  }
}
```

```
<!-- src/app/components/product-list/product-list.component.html -->
<div style="padding: 1.5rem">
  <h2 class="page-title">產品管理</h2>

  <!-- 篩選列 -->
  <div class="filter-bar">
    <input type="text" placeholder="搜尋產品名稱或描述..." [(ngModel)]="keyword" />

    <select [(ngModel)]="selectedLine">
      <option value="">-- 所有產品線 --</option>
      <option *ngFor="let line of productLines" [value]="line">{{ line }}</option>
    </select>

    <select [(ngModel)]="selectedVendor">
      <option value="">-- 所有供應商 --</option>
      <option *ngFor="let v of vendors" [value]="v">{{ v }}</option>
    </select>

    <button (click)="onSearch()">搜尋</button>
    <button class="btn-reset" (click)="onReset()">重置</button>
  </div>

  <p class="result-count">共 {{ products.length }} 筆產品</p>

  <!-- 產品列表 -->
  <table class="data-table">
    <thead>
      <tr>
        <th>產品代碼</th>
        <th>產品名稱</th>
        <th>產品線</th>
        <th>比例</th>
        <th>供應商</th>
        <th class="price-cell">進價 (USD)</th>
        <th class="price-cell">建議售價 (USD)</th>
        <th class="price-cell">毛利率</th>
        <th class="stock-cell">庫存數量</th>
        <th>操作</th>
      </tr>
```

```
    </thead>
    <tbody>
      <tr *ngFor="let p of products">
        <td>{{ p.productCode }}</td>
        <td>{{ p.productName }}</td>
        <td>{{ p.productLine }}</td>
        <td>{{ p.productScale }}</td>
        <td>{{ p.productVendor }}</td>
        <td class="price-cell">{{ p.buyPrice | number:'1.2-2' }}</td>
        <td class="price-cell">{{ p.msrp | number:'1.2-2' }}</td>
        <td class="price-cell">{{ calcMargin(p.buyPrice, p.msrp) }}</td>
        <td class="stock-cell" [class.text-danger]="isLowStock(p.quantityInStock)">
          {{ p.quantityInStock | number }}
        </td>
        <td>
          <a [routerLink]="['/products', p.productCode]" class="action-link">查看</a>
        </td>
      </tr>
      <tr *ngIf="products.length === 0">
        <td colspan="10" style="text-align: center; color: #6c757d;">查無資料</td>
      </tr>
    </tbody>
  </table>
</div>
```

4.6 LowStockAlert Component

```
// src/app/components/low-stock-alert/low-stock-alert.component.ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ProductService } from '../../services/product.service';
import { Product } from '../../models/product.model';
import { StockThreshold } from '../../models/stock-summary.model';

@Component({
  selector: 'app-low-stock-alert',
  standalone: true,
  imports: [CommonModule, FormsModule],
  template: `
    <div style="padding: 1.5rem">
      <h2>低庫存警示管理</h2>

      <!-- 警示說明 -->
      <div class="alert-info">
        <strong>說明：</strong>當產品庫存低於該產品設定的安全水位時，將顯示於此清單。
        預設安全水位為 200 件，建議補貨量為 500 件。
      </div>

      <!-- 低庫存產品列表 -->
      <table class="data-table">
        <thead>
          <tr>
            <th>產品代碼</th>
```

```
        <th>產品名稱</th>
        <th>產品線</th>
        <th>供應商</th>
        <th>目前庫存</th>
        <th>安全水位</th>
        <th>建議補貨量</th>
        <th>危急程度</th>
        <th>操作</th>
      </tr>
    </thead>
    <tbody>
      <tr *ngFor="let p of lowStockProducts">
        <td>{{ p.productCode }}</td>
        <td>{{ p.productName }}</td>
        <td>{{ p.productLine }}</td>
        <td>{{ p.productVendor }}</td>
        <td class="text-right">
          <span [class.critical]="p.quantityInStock < 100"
                [class.warning]="p.quantityInStock >= 100 && p.quantityInStock < 300">
            {{ p.quantityInStock | number }}
          </span>
        </td>
        <td class="text-right">
          <input *ngIf="editingCode === p.productCode"
                 type="number" [(ngModel)]="editMinQty" min="1"
                 style="width:80px; padding: 0.25rem;" />
          <span *ngIf="editingCode !== p.productCode">
            {{ thresholdMap[p.productCode]?.minQuantity ?? 200 | number }}
          </span>
        </td>
        <td class="text-right">
          <input *ngIf="editingCode === p.productCode"
                 type="number" [(ngModel)]="editReorderQty" min="1"
                 style="width:80px; padding: 0.25rem;" />
          <span *ngIf="editingCode !== p.productCode">
            {{ thresholdMap[p.productCode]?.reorderQuantity ?? 500 | number }}
          </span>
        </td>
        <td>
          <span class="badge" [class.badge-danger]="p.quantityInStock < 100"
                              [class.badge-warning]="p.quantityInStock >= 100">
            {{ p.quantityInStock < 100 ? '緊急' : '警示' }}
          </span>
        </td>
        <td>
          <ng-container *ngIf="editingCode === p.productCode; else viewActions">
            <button class="btn btn-save" (click)="saveThreshold(p.productCode)">儲存</button>
            <button class="btn btn-cancel" (click)="cancelEdit()">取消</button>
          </ng-container>
          <ng-template #viewActions>
            <button class="btn" (click)="startEdit(p)">設定水位</button>
          </ng-template>
        </td>
      </tr>
      <tr *ngIf="lowStockProducts.length === 0">
        <td colspan="9" style="text-align:center; color: #28a745; padding: 2rem;">
          ✓ 目前無低庫存警示
        </td>
```

```
          </tr>
        </tbody>
      </table>
    </div>
  `,
  styles: [`
    .alert-info {
      background: #cce5ff; border: 1px solid #b8daff; padding: 1rem;
      border-radius: 4px; margin-bottom: 1.5rem; color: #004085;
    }
    .data-table { width: 100%; border-collapse: collapse; }
    .data-table th, .data-table td { padding: 0.75rem; border-bottom: 1px solid #dee2e6; }
    .data-table th { background: #f8f9fa; }
    .text-right { text-align: right; }
    .critical { color: #dc3545; font-weight: bold; }
    .warning  { color: #ffc107; font-weight: bold; }
    .badge { padding: 0.3rem 0.6rem; border-radius: 4px; font-size: 0.8rem; color: #fff; }
    .badge-danger  { background: #dc3545; }
    .badge-warning { background: #ffc107; color: #000; }
    .btn { padding: 0.3rem 0.7rem; border-radius: 4px; cursor: pointer; border: 1px solid; font-
size: 0.85rem; }
    .btn-save   { background: #28a745; color: #fff; border-color: #28a745; }
    .btn-cancel { background: #6c757d; color: #fff; border-color: #6c757d; margin-left: 4px; }
    .btn:not(.btn-save):not(.btn-cancel) { background: #fff; color: #007bff; border-color: #007bff;
}
  `]
})
export class LowStockAlertComponent implements OnInit {

  lowStockProducts: Product[] = [];
  thresholdMap: { [code: string]: StockThreshold } = {};

  editingCode  = '';
  editMinQty   = 200;
  editReorderQty = 500;

  constructor(private productService: ProductService) {}

  ngOnInit(): void {
    this.loadLowStock();
  }

  loadLowStock(): void {
    this.productService.getLowStockProducts().subscribe(products => {
      this.lowStockProducts = products;
      products.forEach(p => {
        this.productService.getThreshold(p.productCode).subscribe({
          next: t => this.thresholdMap[p.productCode] = t,
          error: () => {}   // 無 threshold 資料則不填入
        });
      });
    });
  }

  startEdit(product: Product): void {
    this.editingCode    = product.productCode;
    this.editMinQty     = this.thresholdMap[product.productCode]?.minQuantity    ?? 200;
    this.editReorderQty = this.thresholdMap[product.productCode]?.reorderQuantity ?? 500;
```

```
  }

  saveThreshold(productCode: string): void {
    this.productService.saveThreshold(productCode, this.editMinQty, this.editReorderQty)
        .subscribe(t => {
          this.thresholdMap[productCode] = t;
          this.cancelEdit();
        });
  }

  cancelEdit(): void { this.editingCode = ''; }
}
```

4.7 路由設定

```
// src/app/app.routes.ts
import { Routes } from '@angular/router';
import { DashboardComponent }      from './components/dashboard/dashboard.component';
import { ProductListComponent }    from './components/product-list/product-list.component';
import { LowStockAlertComponent }  from './components/low-stock-alert/low-stock-alert.component';

export const routes: Routes = [
  { path: '',           redirectTo: 'dashboard', pathMatch: 'full' },
  { path: 'dashboard',  component: DashboardComponent },
  { path: 'products',   component: ProductListComponent },
  { path: 'low-stock',  component: LowStockAlertComponent }
];
```

4.8 根元件（含導覽列）

```
// src/app/app.component.ts
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterModule],
  template: `
    <nav class="navbar">
      <div class="nav-brand">庫存監控系統</div>
      <ul class="nav-links">
        <li><a routerLink="/dashboard"  routerLinkActive="active">儀表板</a></li>
        <li><a routerLink="/products"   routerLinkActive="active">產品管理</a></li>
        <li><a routerLink="/low-stock"  routerLinkActive="active">低庫存警示</a></li>
      </ul>
    </nav>
    <main class="main-content">
      <router-outlet />
    </main>
  `,
```

```
  styles: [`
    .navbar {
      display: flex; align-items: center; justify-content: space-between;
      background: #343a40; color: #fff; padding: 0.75rem 1.5rem;
    }
    .nav-brand { font-size: 1.2rem; font-weight: bold; }
    .nav-links  { list-style: none; display: flex; gap: 1.5rem; margin: 0; padding: 0; }
    .nav-links a { color: #adb5bd; text-decoration: none; }
    .nav-links a:hover, .nav-links a.active { color: #fff; }
    .main-content { max-width: 1200px; margin: 0 auto; }
  `]
})
export class AppComponent {}
```

```
// src/main.ts
import { bootstrapApplication }    from '@angular/platform-browser';
import { provideRouter }            from '@angular/router';
import { provideHttpClient }        from '@angular/common/http';
import { AppComponent }             from './app/app.component';
import { routes }                   from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes),
    provideHttpClient()
  ]
}).catch(err => console.error(err));
```

5. API 規格總表

方法	路徑	說明	對應需求
GET	`/api/products`	查詢全部產品（支援？productLine= &vendor= &keyword=）	FR-01-01 / 02 / 03
GET	`/api/products/{productCode}`	查詢單一產品	FR-01-01
POST	`/api/products`	新增產品	FR-01-01
PUT	`/api/products/{productCode}`	更新產品	FR-01-01
DELETE	`/api/products/{productCode}`	刪除產品	FR-01-01
GET	`/api/products/low-stock`	低庫存警示清單	FR-02-03
GET	`/api/products/dashboard/summary`	儀表板產品線統計	FR-02-05
GET	`/api/products/filter/product-lines`	所有產品線清單	FR-01-02
GET	`/api/products/filter/vendors`	所有供應商清單	FR-01-02
GET	`/api/thresholds/{productCode}`	查詢安全水位設定	FR-02-02
PUT	`/api/thresholds/{productCode}`	新增/更新安全水位	FR-02-02

統一回傳格式

```
{
  "success": true,
  "message": "success",
  "data": { ... }
}
```

錯誤回傳格式

```
{
  "success": false,
  "errorCode": "NOT_FOUND",
```

```
  "message": "Product not found: S99_9999"
}
```

---

6. SQL 初始化腳本

```sql
-- 執行前請確認已連線至 classicmodels 資料庫
USE classicmodels;

-- 建立安全水位資料表
CREATE TABLE IF NOT EXISTS stock_thresholds (
    id               INT           NOT NULL AUTO_INCREMENT PRIMARY KEY,
    product_code     VARCHAR(15)   NOT NULL UNIQUE,
    min_quantity     INT           NOT NULL DEFAULT 200,
    reorder_quantity INT           NOT NULL DEFAULT 500,
    updated_at       DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
                                   ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_threshold_product
        FOREIGN KEY (product_code) REFERENCES products(productCode)
        ON DELETE CASCADE
);

-- 預先為低庫存警示產品設定安全水位
INSERT INTO stock_thresholds (product_code, min_quantity, reorder_quantity) VALUES
    ('S24_2000', 100,  300),   -- 1960 BSA Gold Star DBD34（庫存 15）
    ('S12_1099', 100,  300),   -- 1968 Ford Mustang（庫存 68）
    ('S32_4289', 200,  500),   -- 1928 Ford Phaeton Deluxe（庫存 136）
    ('S32_1374', 200,  500),   -- 1997 BMW F650 ST（庫存 178）
    ('S72_3212', 500, 1000)    -- Pont Yacht（庫存 414）
ON DUPLICATE KEY UPDATE
    min_quantity     = VALUES(min_quantity),
    reorder_quantity = VALUES(reorder_quantity);
```

快速啟動步驟

後端

```bash
# 1. 確認 MySQL classicmodels 資料庫已準備好
# 2. 修改 application.properties 中的密碼
# 3. 啟動 Spring Boot
mvn spring-boot:run
# API 服務啟動於 http://localhost:8080
```

前端

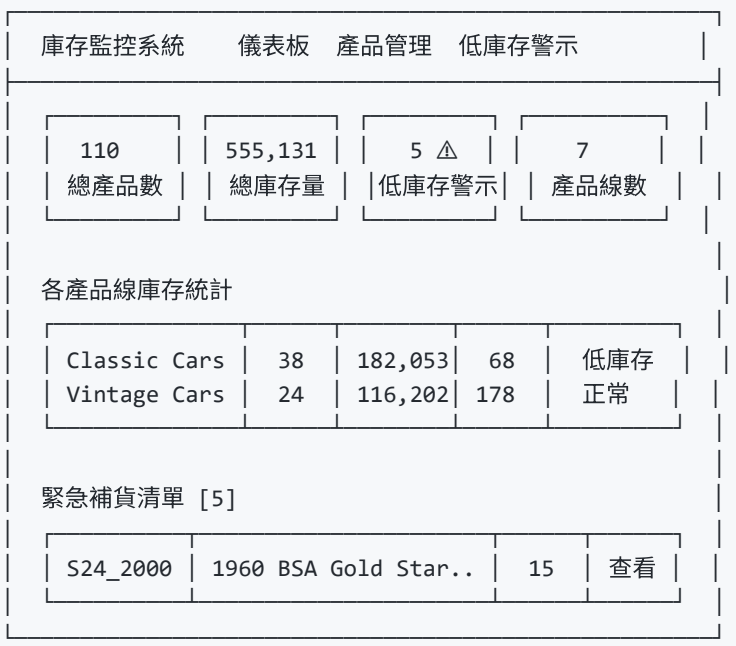
```bash
# 1. 建立 Angular 專案
ng new inventory-frontend --standalone --routing
cd inventory-frontend
```

```
# 2. 複製上述各檔案至對應路徑

# 3. 啟動開發伺服器
ng serve
# 前端啟動於 http://localhost:4200
```

畫面規劃

【儀表板 /dashboard】



【產品管理 /products】
搜尋欄位 | 產品線下拉 | 供應商下拉 | [搜尋] [重置]
表格：代碼 | 名稱 | 產品線 | 供應商 | 進價 | 售價 | 毛利率 | 庫存

【低庫存警示 /low-stock】
表格：代碼 | 名稱 | 庫存 | 安全水位(可編輯) | 建議補貨(可編輯) | 危急程度